

University Timetabling

Creating university timetables is hard, even just to find a feasible plan, where every lecture is given, is hard.

Problem 1

- Create a feasibility model which can generate a feasible timetable

Sets

- Courses: $c \in Courses = \{1, \dots, C\}$
- Lecturers: $l \in Lecturers = \{1, \dots, L\}$
- Rooms: $r \in Rooms = \{1, \dots, R\}$
- Days: $d \in Days = \{1, \dots, D\}$
- Periods: $p \in Periods = \{1, \dots, P\}$
- Curricula: $cu \in Curricula = \{1, \dots, CU\}$

Parameters

- Penalty terms:
 - Missing seat penalty factor: *penalty_missing_seat*
 - Penalty for room changes: *penalty_room_change*
 - Penalty for not teaching a course a minimal no of days: *penalty_less_than_minimum_working_days*
 - Penalty for isolated curricula courses: *penalty_solo_cur_courses*
- No of students in courses: *Course_NoStudents_c*
- No of lectures for course: *Course_NoLectures_c*
- Course lecturer index: *Course_Lecturer_c*
- No of courses for each lecturer: *Lecturer_NoCourses_l*

- Size of rooms: $Rooms_size_r$
- Minimum number of (wished) days for lectures in a course: $Course_MinLectureDays_c$
- Courses in curricula: $Curricula_CourseCurreculaIncidenceMatrix_{cu,c}$.
This is an incidence matrix, where $Curricula_CourseCurreculaIncidenceMatrix_{cu,c} = 1$ means that curricula cu contains course c
- Course placement limitations: $Blocked_course_timeslot_{c,d,p}$, where $Blocked_course_timeslot_{c,d,p} = 1$ means that course c cannot have lectures in day d in period p

Decision variables

- Should a lecture of course c be placed in room r on day d in period p :
 $x_{d,p}^{c,r} \in \{0, 1\}$
- No of seats missing seats in room r in day d in period p : $y_{d,p}^r \geq 0$

Model

Objective:

- Minimize:

42

Constraints:

- Ensure that all lectures are planned:

$$\sum_{r,d,p} x_{d,p}^{c,r} = Course_NoLectures_c \quad \forall c$$

- Ensure at most one course pr room pr day pr timeslot:

$$\sum x_{d,p}^{c,r} \leq 1 \quad \forall r, d, p$$

- Ensures limit the number of students seated in a room, but with a slack variable ensuring feasibility :

$$\sum_c x_{d,p}^{c,r} \leq Rooms_size_r + y_{d,p}^r \quad \forall r, d, p$$

- Ensure that lecturers do not have two or more lectures at the same time

$$\sum_{c|Course_Lecturer_c=l} x_{d,p}^{c,r} \leq 1 \quad \forall l, d, p$$

- Ensure that courses are not taught on blocked days and periods:

$$x_{d,p}^{c,r} = 0 \quad \forall c, r, d, p | \text{Blocked_course_timeslot}_{c,d,p} = 1$$

- Ensure that two courses belonging to the same curriculum, are not assigned to the same day and the same period:

$$\sum_{c,r | \text{Curricula_CourseCurreculaIncidenceMatrix}_{cu,c}=1} x_{d,p}^{c,r} \leq 1 \quad \forall cu, d, p$$

```

*****
# University Timetabling 1, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
using Printf
*****

# PARAMETERS
include("timetable1.jl")
#include("timetable4.jl")
#include("timetable5.jl")
*****

# Model

UTT=Model(HiGHS.Optimizer)
set_time_limit_sec(UTT, 600.0)

# Variables
# 1 if a lecture of course c is placed in room r on day d in period p
@variable(UTT,x[c=1:C,r=1:R,d=1:D,p=1:P],Bin)
for c=1:C
    for d=1:D
        for p=1:P
            if Blocked_course_timeslot[c,d,p]==1
                for r=1:R
                    fix( x[c,r,d,p], 0; force = true)
                end
            end
        end
    end
end
end

```

```

# no of missing seats in room r on day d in period p
@variable(UTT,y[r=1:R,d=1:D,p=1:P] >=0)

# Objective
@objective(UTT, Min, 42)

# Constraints
# ensure that all lectures are planned
@constraint(UTT, [c=1:C],
            sum( x[c,r,d,p] for r=1:R, d=1:D, p=1:P) == Course_NoLectures[c])

# limit the number of students seated in a room,
# but with a slack variable ensuring feasibility
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
            sum( x[c,r,d,p] for c=1:C) <= 1)

# no double lectures for lecturereres
@constraint(UTT, [l=1:L,d=1:D,p=1:P; Lecturer_NoCourses[l]>1],
            sum( x[c,r,d,p] for c=1:C, r=1:R if Course_Lecturer[c] == l) <= 1)

# set the objective variabel y[r,d,p] according to the number of seats missing
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
            sum( Course_NoStudents[c]*x[c,r,d,p] for c=1:C) <= Rooms_size[r] + y[r,d,p])

# if Blocked_course_timeslot[c,d,p]==1,
# ensure that course c is not taught on day d in timeslot p
@constraint(UTT, blocked_day_period_for_course[c=1:C,d=1:D,p=1:P;
            Blocked_course_timeslot[c,d,p]==1],
            sum( x[c,r,d,p] for r=1:R) == 0)

# ensure that two courses belonging to the same curriculum
# are not assigned same day and timeslot
@constraint(UTT, [cu=1:CU,d=1:D,p=1:P],
            sum( x[c,r,d,p] for c=1:C, r=1:R if
            Curricula_CourseCurreculaIncidenceMatrix[cu,c]==1) <= 1)

#####

*****
# solve
optimize!(UTT)
*****

*****
# Report results
println("-----");

```

```

if termination_status(UTT) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(UTT))")
else
    println(" No solution")
end
println("-----");
#####

```

Comments to the Julia/JuMP program:

- Notice: We have simply put a constant in the objective function. We are only looking for a feasible solution.
- Notice: We fix the blocked variables to 0, not using a constraint

Problem 2

Minimize the penalty for missing seats in the timetable. This only requires to update the objective function, to include the $y_{d,p}^r$ slack variable.

- Minimize the total number of students who do not have a seat in the lectures.

New objective function:

Objective:

- Minimize:

$$penalty_missing_seat \cdot \sum_{r,d,p} y_{d,p}^r$$

```

#####
# University Timetabling 2, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
using Printf
#####

#####
# PARAMETERS
include("timetable1.jl")
#include("timetable4.jl")
#include("timetable5.jl")
#####

```

```

*****
# Model

UTT=Model(HiGHS.Optimizer)
set_time_limit_sec(UTT, 600.0)

# Variables
# 1 if a lecture of course c is placed in room r on day d in period p
@variable(UTT,x[c=1:C,r=1:R,d=1:D,p=1:P],Bin)
for c=1:C
    for d=1:D
        for p=1:P
            if Blocked_course_timeslot[c,d,p]==1
                for r=1:R
                    fix( x[c,r,d,p], 0; force = true)
                end
            end
        end
    end
end

# no of missing seats in room r on day d in period p
@variable(UTT,y[r=1:R,d=1:D,p=1:P] >=0)

# Objective
@objective(UTT, Min,
    penalty_missing_seat*sum( y[r,d,p] for r=1:R, d=1:D, p=1:P))

# Constraints
# at most one course pr room pr day pr timeslot
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
    sum( x[c,r,d,p] for c=1:C) <= 1)

# no double lectures for lecturereres
@constraint(UTT, [l=1:L,d=1:D,p=1:P; Lecturer_NoCourses[l]>1],
    sum( x[c,r,d,p] for c=1:C, r=1:R if Course_Lecturer[c] == l) <= 1)

# ensure that all lectures are planned
@constraint(UTT, [c=1:C],
    sum( x[c,r,d,p] for r=1:R, d=1:D, p=1:P) == Course_NoLectures[c])

# set the objective variabel y[r,d,p] according to the number of seats missing
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
    sum( Course_NoStudents[c]*x[c,r,d,p] for c=1:C) <= Rooms_size[r] + y[r,d,p])

# if Blocked_course_timeslot[c,d,p]==1,

```

```

# ensure that course c is not taught on day d in timeslot p
@constraint(UTT, blocked_day_period_for_course[c=1:C,d=1:D,p=1:P;
    Blocked_course_timeslot[c,d,p]==1],
    sum( x[c,r,d,p] for r=1:R) == 0)

# ensure that two courses belonging to the same curriculum
# are not assigned same day and timeslot
@constraint(UTT, [cu=1:CU,d=1:D,p=1:P],
    sum( x[c,r,d,p] for c=1:C, r=1:R
    if Curricula_CourseCurriculaIncidenceMatrix[cu,c]==1) <= 1)

#####

*****
# solve
optimize!(UTT)
*****

*****
# Report results
println("-----");
if termination_status(UTT) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(UTT))")
else
    println(" No solution")
end
*****

```

Problem 3

Now we want to improve the room stability, i.e. each course use as few rooms as possible, preferably just one room pr. course.

To do this, we introduce two new variables u_r^c and u_{room}^c and two new constraints to set these variable and finally a objective function. First the variables then the new objective function:

Decision variables

- Is course c using room r : $u_r^c \in \{0,1\}$

- How many rooms, more than one is used for lectures in course c : $u_room^c \in R^+$

Assuming that the new variables are correctly adjusted through the constraints further below, the objective function has another sum added:

- Minimize:

$$penalty_missing_seat \cdot \sum_{r,d,p} y_{d,p}^r + penalty_room_change \cdot \sum_c u_room^c$$

Constraints:

- Ensure that the u_r^c variable is 1 if course c use room r :

$$\sum_{d,p} x_{d,p}^{c,r} \leq Course_NoLectures_c \cdot u_r^c \cdot u_r^c \quad \forall c, r$$

- Force the value of u_room^c variable to be equal to the number of extra rooms necessary:

$$u_room^c \geq \sum_r \sum_{d,p} x_{d,p}^{c,r} \leq (Course_NoLectures_c \cdot u_r^c) - 1 \cdot u_r^c \quad \forall c, r$$

```

*****
# University Timetabling 3, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
using Printf
*****

*****
# PARAMETERS
include("timetable1.jl")
#include("timetable4.jl")
#include("timetable5.jl")
*****

*****
# Model

UTT=Model(HiGHS.Optimizer)
#set_silent(UTT)

# Variables

```

```

# 1 if a lecture of course c is placed in room r on day d in period p
@variable(UTT,x[c=1:C,r=1:R,d=1:D,p=1:P],Bin)
for c=1:C
    for d=1:D
        for p=1:P
            if Blocked_course_timeslot[c,d,p]==1
                for r=1:R
                    fix( x[c,r,d,p], 0; force = true)
                end
            end
        end
    end
end

# no of missing seats in room r on day d in period p
@variable(UTT,y[r=1:R,d=1:D,p=1:P] >=0)

# 1 if room r is used by a lecture of course c
@variable(UTT,u[c=1:C,r=1:R],Bin)

# no of rooms more than 1 needed by course c
@variable(UTT,u_room[c=1:C] >=0)

# Objective
@objective(UTT, Min,
    penalty_missing_seat*sum( y[r,d,p] for r=1:R, d=1:D, p=1:P) +
    penalty_room_change*sum( u_room[c] for c=1:C)
)

# Constraints
# at most one course pr room pr day pr timeslot
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
    sum( x[c,r,d,p] for c=1:C) <= 1)

# no double lectures for lectureres
@constraint(UTT, [l=1:L,d=1:D,p=1:P; Lecturer_NoCourses[l]>1],
    sum( x[c,r,d,p] for c=1:C, r=1:R if Course_Lecturer[c] == l) <= 1)

# ensure that all lectures are planned
@constraint(UTT, [c=1:C],
    sum( x[c,r,d,p] for r=1:R, d=1:D, p=1:P) == Course_NoLectures[c])

# set the objective variabel y[r,d,p] according to the number of seats missing
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
    sum( Course_NoStudents[c]*x[c,r,d,p] for c=1:C) <= Rooms_size[r] + y[r,d,p])

```

```

# if Blocked_course_timeslot[c,d,p]==1,
# ensure that course c is not taught on day d in timeslot p
@constraint(UTT, blocked_day_period_for_course[c=1:C,d=1:D,p=1:P;
    Blocked_course_timeslot[c,d,p]==1],
    sum( x[c,r,d,p] for r=1:R) == 0)

# ensure that two courses belonging to the same curriculum
# are not assigned same day and timeslot
@constraint(UTT, [cu=1:CU,d=1:D,p=1:P],
    sum( x[c,r,d,p] for c=1:C, r=1:R if
    Curricula_CourseCurriculaIncidenceMatrix[cu,c]==1) <= 1)

# force the value of the u[c,r] variable to 1 if course c use room r
@constraint(UTT, force_u_use_of_room_for_course[c=1:C,r=1:R],
    sum( x[c,r,d,p] for d=1:D, p=1:P) <= Course_NoLectures[c]*u[c,r])

# count the number of rooms used by course c, minus 1
@constraint(UTT, [c=1:C], u_room[c] >= sum( u[c,r] for r=1:R) - 1)
#####

#####
# solve
set_time_limit_sec(UTT, 600)
optimize!(UTT)
#####

#####
# Report results
println("-----");
if termination_status(UTT) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(UTT))")
    println("Combined seat objective penalty: ",
        penalty_missing_seat*sum( value(y[r,d,p]) for r=1:R, d=1:D, p=1:P))
    println("Combined room change penalty: ",
        penalty_room_change*sum( value(u_room[c]) for c=1:C))
else
    println(" No solution")
end
println("-----");
#####

```

Problem 4

Now we want to spread the lectures in the different courses more, such that if a course has a $Course_MinLectureDays_c = 3$, if all the lectures of the course is on the same day, then there is a penalty of $2 \cdot penalty_less_than_minimum_working_days$. To achieve this, two new variables wd_d^c and nwd_c are declared to calculate this using two new constraints and a new term is added to the objective

Decision variables

- Is course c taught on day d : $wd_d^c \in \{0, 1\}$
- How many days is missing in the teaching of course c : $nwd_c \in R^+$

Assuming that the new variables are correctly adjusted through the constraints further below, the objective function has another sum added:

- Minimize:

$$penalty_missing_seat \cdot \sum_{r,d,p} y_{d,p}^r +$$

$$penalty_room_change \cdot \sum_c u_room^c +$$

$$penalty_less_than_minimum_working_days \cdot \sum_c nwd_c$$

New constraints:

- Set the variable wd_d^c :

$$\sum_{r,p} x_{d,p}^{c,r} \leq u_r^c \cdot wd_d^c \quad \forall c, d$$

- Calculate the number of missing teaching days nwd_c :

$$nwd_c \geq Course_MinLectureDays_c - \sum_d wd_d^c \quad \forall c \quad \sum_{r,p} x_{d,p}^{c,r} \leq P \cdot u_r^c \cdot wd_d^c \quad \forall c, d$$

```
*****
# University Timetabling 4, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
using Printf
*****

*****
```

```

# PARAMETERS
include("timetable1.jl")
#include("timetable4.jl")
#include("timetable5.jl")
*****

*****
# Model

UTT=Model(HiGHS.Optimizer)
#set_silent(UTT)
set_time_limit_sec(UTT, 600.0)

# Variables
# 1 if a lecture of course c is placed in room r on day d in period p
@variable(UTT,x[c=1:C,r=1:R,d=1:D,p=1:P],Bin)
for c=1:C
    for d=1:D
        for p=1:P
            if Blocked_course_timeslot[c,d,p]==1
                for r=1:R
                    fix( x[c,r,d,p], 0; force = true)
                end
            end
        end
    end
end
end

# no of missing seats in room r on day d in period p
@variable(UTT,y[r=1:R,d=1:D,p=1:P] >=0)

# 1 if room r is used by a lecture of course c
@variable(UTT,u[c=1:C,r=1:R],Bin)

# no of rooms more than 1 needed by course c
@variable(UTT,u_room[c=1:C] >=0)

# 1 if course c has one or more lectures on day d
@variable(UTT,0 <= wd[c=1:C,d=1:D] <=1)

# no of days LESS than the minimum number of working days
@variable(UTT,nwd[c=1:C] >=0)

# Objective
@objective(UTT, Min,
    penalty_missing_seat*sum( y[r,d,p] for r=1:R, d=1:D, p=1:P) +

```

```

        penalty_room_change*sum( u_room[c] for c=1:C ) +
        penalty_less_than_minimum_working_days*sum( nwd[c] for c=1:C )
    )

# Constraints
# at most one course pr room pr day pr timeslot
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
    sum( x[c,r,d,p] for c=1:C ) <= 1)

# no double lectures for lecturereres
@constraint(UTT, [l=1:L,d=1:D,p=1:P; Lecturer_NoCourses[l]>1],
    sum( x[c,r,d,p] for c=1:C, r=1:R if Course_Lecturer[c] == l ) <= 1)

# ensure that all lectures are planned
@constraint(UTT, [c=1:C],
    sum( x[c,r,d,p] for r=1:R, d=1:D, p=1:P ) == Course_NoLectures[c])

# set the objective variabel y[r,d,p] according to the number of seats missing
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
    sum( Course_NoStudents[c]*x[c,r,d,p] for c=1:C ) <= Rooms_size[r] + y[r,d,p])

# if Blocked_course_timeslot[c,d,p]==1,
# ensure that course c is not taught on day d in timeslot p
@constraint(UTT, blocked_day_period_for_course[c=1:C,d=1:D,p=1:P;
    Blocked_course_timeslot[c,d,p]==1],
    sum( x[c,r,d,p] for r=1:R ) == 0)

# ensure that two courses belonging to the same curriculum
# are not assigned same day and timeslot
@constraint(UTT, [cu=1:CU,d=1:D,p=1:P],
    sum( x[c,r,d,p] for c=1:C, r=1:R
    if Curricula_CourseCurreculaIncidenceMatrix[cu,c]==1 ) <= 1)

# force the value of the u[c,r] variable to 1 if course c use room r
@constraint(UTT, [c=1:C,r=1:R],
    sum( x[c,r,d,p] for d=1:D, p=1:P ) <= Course_NoLectures[c]*u[c,r])

# count the number of rooms used by course c, minus 1
@constraint(UTT, [c=1:C], u_room[c] >= sum( u[c,r] for r=1:R ) - 1)

# allow wd to become 1 if at least one lecture of course c is on day d
@constraint(UTT, [c=1:C,d=1:D],
    sum( x[c,r,d,p] for r=1:R, p=1:P ) >= wd[c,d])

# force nwd penalty variable to achieve the difference up to MinLectureDays
@constraint(UTT, [c=1:C],

```

```

nwd[c] >= Course_MinLectureDays[c] - sum( wd[c,d] for d=1:D))
#####

#####

# solve
set_time_limit_sec(UTT, 600)
optimize!(UTT)
#####

#####

# Report results
println("-----");
if termination_status(UTT) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(UTT))")
    println("Combined seat objective penalty: ",
            penalty_missing_seat*sum( value(y[r,d,p]) for r=1:R, d=1:D, p=1:P))
    println("Combined room change penalty: ",
            penalty_room_change*sum( value(u_room[c]) for c=1:C))

    println("Combined minimum working days penalty: ",
            penalty_less_than_minimum_working_days*sum( value(nwd[c]) for c=1:C ))
else
    println(" No solution")
end
println("-----");
#####

```

Problem 5

Finally, we want the model to minimize the number of isolated lectures for each curriculum. This model we will describe below, **but** this constraint makes the model very hard to solve. The three datasets available, `timetable1.jl`, `timetable4.jl` and `timetable5.jl`, can all be solved using Gurobi, but HiGHS struggles. Of the other 18 datasets only few can be solved to optimality.

Decision variables

- Is a course from curriculum cu taught on day d in period p : $cuc_{d,p}^{cu} \in [0, 1]$
- Is a course from curriculum cu isolated on day d in period p : $cus_{d,p}^{cu} \in [0, 1]$

Assuming that the new variables are correctly adjusted through the constraints further below, the objective function has another sum added:

- Minimize:

$$\begin{aligned}
& \text{penalty_missing_seat} \cdot \sum_{r,d,p} y_{d,p}^r + \\
& \text{penalty_room_change} \cdot \sum_c u_room^c + \\
& \text{penalty_less_than_minimum_working_days} \cdot \sum_c nwd_c + \\
& \text{penalty_solo_cur_courses} \cdot \sum_{cu,d,p} cus_{d,p}^{cu}
\end{aligned}$$

New constraints:

- The previous constraint used to avoid two or more courses from the same curriculum on the same day and period, is changed such that the $cuc_{d,p}^{cu}$ is on the right-hand-side.

$$\sum_{c,r | \text{Curricula_CourseCurreculaIncidenceMatrix}_{cu,c}=1} x_{d,p}^{c,r} \leq cuc_{d,p}^{cu} \quad \forall cu, d, p$$

- Now a constraint compares each timeslot with the neighbour timeslots:

$$cuc_{d,p}^{cu} - cuc_{d,p+1}^{cu} | (p < P) - cuc_{d,p-1}^{cu} | (p > 1) \leq cus^{cu,d,p} \quad \forall cu, d, p$$

```

*****
# University Timetabling 5, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
using Gurobi
using Printf
*****

# PARAMETERS
#include("timetable1.jl")
#include("timetable4.jl")
include("timetable5.jl")
*****

# Model

UTT=Model(HiGHS.Optimizer)
UTT=Model(Gurobi.Optimizer)
#set_silent(UTT)

```

```

# Variables
# 1 if a lecture of course c is placed in room r on day d in period p
@variable(UTT,x[c=1:C,r=1:R,d=1:D,p=1:P],Bin)
for c=1:C
    for d=1:D
        for p=1:P
            if Blocked_course_timeslot[c,d,p]==1
                for r=1:R
                    fix( x[c,r,d,p], 0; force = true)
                end
            end
        end
    end
end

# no of missing seats in room r on day d in period p
@variable(UTT,y[r=1:R,d=1:D,p=1:P] >=0)

# 1 if room r is used by a lecture of course c
@variable(UTT,u[c=1:C,r=1:R],Bin)

# no of rooms more than 1 needed by course c
@variable(UTT,u_room[c=1:C] >=0)

# 1 if course c has one or more lectures on day d
@variable(UTT,0 <= wd[c=1:C,d=1:D] <=1)

# no of days LESS than the minimum number of working days
@variable(UTT,nwd[c=1:C] >=0)

# 1 if curriculum cu has a course on day d in period p
@variable(UTT,0 <= cuc[cu=1:CU,d=1:D,p=1:P] <=1)

# 1 if isolated lecture of curriculum cu on day d in period p
@variable(UTT,0 <= cus[cu=1:CU,d=1:D,p=1:P] <=1)

# Objective
@objective(UTT, Min,
    penalty_missing_seat*sum( y[r,d,p] for r=1:R, d=1:D, p=1:P) +
    penalty_room_change*sum( u_room[c] for c=1:C) +
    penalty_less_than_minimum_working_days*sum( nwd[c] for c=1:C ) +
    penalty_solo_cur_courses*sum( cus[cu,d,p] for cu=1:CU, d=1:D, p=1:P)
)

# Constraints

```

```

# at most one course pr room pr day pr timeslot
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
    sum( x[c,r,d,p] for c=1:C) <= 1)

# no double lectures for lecturereres
@constraint(UTT, [l=1:L,d=1:D,p=1:P; Lecturer_NoCourses[l]>1],
    sum( x[c,r,d,p] for c=1:C, r=1:R if Course_Lecturer[c] == l) <= 1)

# ensure that all lectures are planned
@constraint(UTT, [c=1:C],
    sum( x[c,r,d,p] for r=1:R, d=1:D, p=1:P) == Course_NoLectures[c])

# set the objective variabel y[r,d,p] according to the number of seats missing
@constraint(UTT, [r=1:R,d=1:D,p=1:P],
    sum( Course_NoStudents[c]*x[c,r,d,p] for c=1:C) <= Rooms_size[r] + y[r,d,p])

# if Blocked_course_timeslot[c,d,p]==1,
# ensure that course c is not taught on day d in timeslot p
@constraint(UTT, blocked_day_period_for_course[c=1:C,d=1:D,p=1:P;
    Blocked_course_timeslot[c,d,p]==1],
    sum( x[c,r,d,p] for r=1:R) == 0)

# ensure that two courses belonging to the same curriculum
# are not assigned same day and timeslot
@constraint(UTT, [cu=1:CU,d=1:D,p=1:P],
    sum( x[c,r,d,p] for c=1:C, r=1:R
    if Curricula_CourseCurreculaIncidenceMatrix[cu,c]==1) == cuc[cu,d,p])

# force the value of the u[c,r] variable to 1 if course c use room r
@constraint(UTT, [c=1:C,r=1:R],
    sum( x[c,r,d,p] for d=1:D, p=1:P) <= Course_NoLectures[c]*u[c,r])

# count the number of rooms used by course c, minus 1
@constraint(UTT, [c=1:C], u_room[c] >= sum( u[c,r] for r=1:R) - 1)

# allow wd to become 1 if at least one lecture of course c is on day d
@constraint(UTT, [c=1:C,d=1:D],
    sum( x[c,r,d,p] for r=1:R, p=1:P) >= wd[c,d])

# force nwd penalty variable to achieve the difference up to MinLectureDays
@constraint(UTT, [c=1:C],
    nwd[c] >= Course_MinLectureDays[c] - sum( wd[c,d] for d=1:D))

# force cus variable to 1 if course in curriculum cu on day d in periodp is isolated
@constraint(UTT, force_cus_penalty[cu=1:CU,d=1:D,p=1:P],

```

```

        cuc[cu,d,p] - (p<P ? cuc[cu,d,p+1] : 0) - (p>1 ? cuc[cu,d,p-1] : 0)
        <= cus[cu,d,p])
#####

#####

# solve
set_time_limit_sec(UTT, 600)
optimize!(UTT)
#####

#####

# Report results
println("-----");
if termination_status(UTT) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(UTT))")
    println("Combined seat objective penalty: ",
        penalty_missing_seat*sum( value(y[r,d,p]) for r=1:R, d=1:D, p=1:P))
    println("Combined room change penalty: ",
        penalty_room_change*sum( value(u_room[c]) for c=1:C))
    println("Combined minimum working days penalty: ",
        penalty_less_than_minimum_working_days*sum( value(nwd[c]) for c=1:C ))
    println("Combined curriculum isolated penalty: ",
        penalty_solo_cur_courses*sum( value(cus[cu,d,p]) for cu=1:CU, d=1:D, p=1:P))
else
    println(" No solution")
end
#####

```